URBAN AUTOMATED UNDERGROUND WASTE PNEUMATIC TRANSPORT HUB

JAVA PROJECT
NOVEMBER 2025

JAVA PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF ENGINEERING** IN **COMPUTER SCIENCE AND ENGINEERING** OF THE ANNA UNIVERSITY

PROJECT WORK

THIRULOGESH M-722825104234

BATCH
2024 – 2028

Under the Guidance of
Mr. M. KARTHICK RAJA M.E.,
ASSISTANT PROFESSOR

Department of Computer Science & Engineering
Sri Eshwar College of Engineering
(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

Sri Eshwar College of Engineering

(An Autonomous Institution – Affiliated to Anna University)

COIMBATORE – 641 202

BONAFIDE CERTIFICATE

Certif Certified that this Report titled " **Urban Automated Underground Waste Pneumatic Transport Hub** e " is the bonafide work of

bonafide work of

THIRULOGESH M **722825104234**

who carried out the project work under my supervision.

SIGNATURE

Dr. R. SUBHA M.E., Ph.D.

PROFESSOR

HEAD OF THE DEPARTMENT

Computer Science and Engineering,

Sri Eshwar College of Engineering,

Coimbatore – 641 202.

SIGNATURE

Mr. M. KARTHICK RAJA M.E.,

SUPERVISOR

Assistant Professor,

Computer Science and Engineering,

Sri Eshwar College of Engineering,

Coimbatore – 641 202.

Submitted for the **Autonomous Semester End Java Project Viva-Voce** held on

……..

INTERNAL EXAMINER **EXTERNAL EXAMINER**

QUALITY POLICY

To develop and implement reliable, automated, and environmentally sustainable engineering solutions that modernize urban sanitation through automated underground pneumatic waste conveyance, real-time pipeline telemetry, smart collection hub monitoring, and intelligent transport coordination, while continuously improving system quality, reliability, and operational efficiency.

INSTITUTE VISION

To develop technologically proficient, socially responsible, and innovative engineering professionals capable of creating intelligent digital and infrastructural solutions for real-world challenges in urban sustainability, automation, and smart cities.

INSTITUTE MISSION

- Providing outcome and value-based engineering education with emphasis on practical and technology-oriented learning.
- Nurturing research, automation, and entrepreneurial culture for real-world municipal and industrial challenges.
- Enabling students to be industry ready and fulfill their career aspirations in modern software and IoT automation.
- Grooming students through behavioral and leadership training programs.
- Making students socially and environmentally responsible.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT VISION

To develop competent computer science professionals capable of designing secure, innovative, and intelligent software and automated solutions for smart city infrastructure and modern computing applications.

DEPARTMENT MISSION

- Providing students with strong foundations in programming, software engineering, databases, and modern computing technologies.
- Encouraging students to develop innovative software solutions for real-world urban and environmental problems using emerging technologies.
- Providing practical exposure to technologies such as Java, Spring Boot, REST APIs, databases, telemetry systems, and web application development.

- Developing analytical thinking, teamwork, leadership, communication, and full-stack software development skills.
- Generating opportunities for students to evolve as competent software professionals with societal and ecological consciousness.

PROGRAM EDUCATIONAL OBJECTIVES

- **PEO1:** Graduates will build successful careers in computing and automation domains by applying programming, database management, software engineering, and IoT principles to real-world industrial and civic applications.
- **PEO2:** Graduates will continuously learn and adapt to emerging technologies such as pneumatic transport automation, sensor-based monitoring, cloud telematics, and intelligent urban infrastructure.
- **PEO3:** Graduates will design, implement, and maintain innovative, secure, ethical, and ecologically sustainable computing solutions addressing challenges in automated waste collection, urban hygiene, resource recovery, and smart city operations.

PROGRAM OUTCOMES

PO1: Engineering knowledge:
Apply the knowledge of mathematics, computer science, programming, databases, and software engineering to the solution of complex municipal waste pneumatic conveyance and telemetry problems.

PO2: Problem Analysis:
Identify, formulate, research literature and analyze engineering challenges related to underground pneumatic pipeline blockages, differential air-pressure regulation, waste container overflow, and multi-point transport routing.

PO3: Design/Development of Solutions:
Design and develop robust software and telemetry systems for pneumatic valve cycling, automated suction sequencing, bin fill-level alerts, emergency depressurization, and centralized hub operation.

PO4: Conduct investigations of complex problems:
Investigate pipeline airflow dynamics, acoustic blockage signatures, load distribution, and transport cycle energy consumption using data analytics to optimize throughput.

PO5: Modern Tool Usage:
Select and apply modern computing tools and frameworks including Java, Spring Boot, MySQL, REST APIs, HTML5/CSS3, JavaScript, Maven, Git, and Postman for system modeling, control, and verification.

PO6: The Engineer and Society:
Apply reasoning informed by contextual knowledge to assess civic hygiene, public health, worker occupational safety, and regulatory compliance in automated underground waste management.

PO7: Environment and Sustainability:
Understand the environmental impact of eliminating diesel garbage trucks, reducing surface carbon emissions, mitigating odor pollution, and advancing circular economy goals through pneumatic waste transit.

PO8: Ethics:
Apply ethical principles and commit to professional standards in data integrity, environmental stewardship, public safety compliance, and responsible infrastructural automation.

PO9: Individual and Team Work:
Function effectively as an individual and as a member or leader in diverse engineering teams across requirements analysis, software architecture, backend service implementation, and testing.

PO10: Communication:
Communicate technical information effectively through structured system documentation, architectural blueprints, database schemas, API specifications, and formal project reports.

PO11: Project Management and Finance:
Demonstrate knowledge of engineering and financial management to plan development milestones, resource allocation, operational expenditure, and lifecycle cost estimation for municipal automated hubs.

PO12: Life-long Learning:
Recognize the need for continuous technological learning to adapt to advances in pneumatic logistics, urban robotics, sensor telemetry, and intelligent smart-city utility frameworks.

PROGRAM SPECIFIC OUTCOMES

PSO1. Demonstrate practical proficiency in modern open-source stacks including Java, Spring Boot, MySQL, RESTful Web Services, and web technologies to develop resilient automation applications.

PSO2. Develop innovative software solutions incorporating IoT sensor monitoring, automated pneumatic control logic, and real-time operational registers for smart city municipal infrastructure.

PSO3. Apply systematic Software Development Life Cycle principles from requirement gathering and architectural modeling to database persistence, testing, and deployment of mission-critical systems.

DECLARATION

THIRULOGESH M [722825104234]

To declare that the project entitled **"Urban Automated Underground Waste Pneumatic Transport Hub"** submitted in partial fulfilment to the University as the project work of Bachelor of Engineering (Computer Science and Engineering) Degree, is a record of original work done by us under the supervision and guidance of **Mr. M. KARTHICK RAJA M.E.,** Assistant Professor, Department of Computer Science and Engineering, Sri Eshwar College of Engineering, Coimbatore.

Place: Coimbatore
Date:

THIRULOGESH M
722825104234

Project Guided by,

Mr. M. KARTHICK RAJA M.E., AP/CSE

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. I take this opportunityto express my gratitude and thanks to everyone who helped me in my project. I would like to thank the management for the constant support provided by them to complete this project.

It is indeed our great honor bounded duty to thank our beloved **Chairman Mr. R.Mohanram,** for his academic interest shown towards the students.

We are indebted to our **Director Mr. R. Rajaram,** for motivating and providing uswith all facilities.

I wish to express my sincere regards and deep sense of gratitude to

Dr. Sudha Mohanram, M.E, Ph.D. Principal, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr. R. Subha, M.E., Ph.D.** Head of Computer Science and Engineering Department for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department.

I express my sincere thanks to my project Coordinator **Mr. M. Karthick Raja M.E.,** Assistant Professors of Computer Science and Engineering Department for the valuable guidance and encouragement given, to us for this project.

I solemnly express our thanks to all the teaching and nonteaching staff of the Computer Science and Engineering Department, family and friends for their valuable support which inspired us to work on this project.

TABLE OF CONTENT

CONTENT	PAGE NO
ABSTRACT	1
LIST OF FIGURES	2
LIST OF TABLE	3
1. INTRODUCTION	
1.1. PROJECT OVERVIEW & OBJECTIVES	4
1.2. PROBLEM STATEMENT AND MOTIVATION	5
2. SYSTEM REQUIREMENTS & SPECIFICATION	
2.1. FUNCTIONAL REQUIREMENTS	6
2.2. NON-FUNCTIONAL REQUIREMENTS	7
3. LOW-LEVEL DESIGN (LLD)	
3.1. SYSTEM ARCHITECTURE DESIGN	9
3.2. COMPONENT DESIGN &MODULE STRUCTURE	10
4. MODELING (UML DIAGRAMS)	
4.1. USE CASE DIAGRAMS	12
4.2. CLASS DIAGRAMS AND SEQUENCE DIAGRAMS	13
4.3. FLOW DIAGRAMS AND ACTIVITY DIAGRAMS	15
5. DATABASE DESIGN	
5.1. ENTITY RELATIONSHIP DIAGRAM	17
5.2. DATABASE SCHEMA AND TABLE STRUCTURE	28
6. IMPLEMENTATION AND RESULT	
6.1. FILES WITH CODES	20
6.2. RESULT	26
7. CONCLUSION AND FUTURE SCOPE	
7.1. PROJECT SUMMARY AND ACHIEVEMENTS	28
7.2. FUTURE ENHANCEMENT AND SCOPE	29
8. APPENDICES AND REFERENCES	
8.1. REFERENCES	30
8.2. APENDIX A-SOURCE CODE STRUCTURE	30
8.3. APPENDIX B	72

ABSTRACT

The Urban Automated Underground Waste Pneumatic Transport Hub project is an advanced, sustainable municipal waste management platform and backend control system engineered to modernize city sanitation and solid waste logistics. Traditional municipal waste collection depends on surface trash bins, manual sorting, and heavy diesel garbage trucks, which create traffic congestion, odor pollution, public health hazards, and high operational costs. This project implements a fully automated, underground pneumatic waste transit network coupled with IoT sensor telematics, pneumatic booster station control, and real-time inventory and disposal tracking.

Built using the Spring Boot framework with MySQL database integration, the system provides automated container fill-level monitoring, multi-point pneumatic pipeline suction sequencing, fault detection, and real-time operational ledger accounting. The backend architecture incorporates deterministic concurrency controls, JPA/Hibernate persistence, and responsive operator dashboards, enabling municipal authorities to monitor waste volume, pipeline airflow velocity, scheduled pneumatic transport cycles, and disposal receipts seamlessly.

The platform eliminates above-ground waste accumulation, significantly reduces urban carbon emissions, and ensures a clean, smart, and hygienic urban environment, demonstrating a robust and scalable architectural model for contemporary smart city infrastructure.

LIST OF FIGURES

Fig No	NAME OF THE FIGURE	Page No
3.1	SYSTEM ARCHITECTURE	9
3.2.1	MODULE STRUCTURE	10
3.2.2	COMPONENT DESIGN	11
4.1	USE CASE DIAGRAM	12
4.2.1	CLASS DIAGRAM	13
4.2.2	SEQUENCE DIAGRAM	14
4.3.1	FLOW DIAGRAM	15
4.3.2	ACTIVITY DIAGRAM	16
5.1	ER DIAGRAM	17
5.2	DATABASE SCHEMA	18
6.2.1	LOGIN/REGISTER PAGE	26
6.2.2	HOME PAGE	26
6.2.3	ADMIN DASHBOARD	27

LIST OF TABLES

Table. No	NAME OF THE TABLE	Page No
5.2.1	USERS TABLE	18
5.2.2	STORIES TABLE	19
5.2.3	LIKES TABLES	19
5.2.4	COMMENT TABLES	19

1. INTRODUCTION

1.1 PROJECT OVERVIEW AND OBJECTIVES

The Campus Mentorship Stories project is a web-based platform designed to foster knowledge sharing and community building within academic institutions. This digital storytelling platform serves as a centralized hub where students, teachers, and mentors can share their experiences, insights, and inspirational stories related to academic growth and career development. The application implements a robust user management system with role-based authentication, ensuring secure access while maintaining appropriate permissions for different user categories. Built using Spring Boot, MySQL, and Thymeleaf, the platform provides an intuitive and responsive user interface that encourages active participation among the campus community. The primary objectives include creating a digital space for sharing mentorship experiences, enabling users to document and share valuable insights, implementing secure authentication with distinct roles for adJavastrators, students, teachers, and mentors, and providing comprehensive story creation and management capabilities. Secondary objectives focus on enabling community interaction through likes and comments, implementing robust admin panel for user and content moderation, and designing a maintainable system using industry-standard frameworks. The system supports role-based access control where users can register with specific roles while adJavastrators maintain oversight through a dedicated panel. Users can create categorized stories with optional author attribution and engage with content through interactive features, promoting collaborative learning and mentorship culture within educational institutions

1.2 PROBLEM STATEMENT AND MOTIVATION

In today's academic environment, valuable mentorship experiences and success stories often remain isolated within individual interactions, limiting their potential impact on the broader campus community. Students frequently struggle to find relevant guidance and inspiration from peers, teachers, and mentors who have faced similar challenges, while experienced individuals lack a centralized platform to share their knowledge and insights. Traditional communication methods such as informal conversations, emails, or bulletin boards fail to create a sustainable repository of mentorship wisdom that can benefit future generations of students. The absence of a structured digital platform results in the loss of valuable experiences, missed opportunities for community building, and inefficient knowledge transfer within educational institutions. Furthermore, existing social media platforms are not specifically designed for academic mentorship purposes and lack the necessary features for categorizing, searching, and managing educational content effectively. The motivation for developing the Campus Mentorship Stories platform stems from the need to bridge this gap by creating a dedicated space where the campus community can document, share, and discover meaningful mentorship experiences. This digital solution aims to preserve institutional knowledge, foster stronger connections between different user groups, and create a culture of continuous learning and support. By implementing role-based access control and interactive features, the platform ensures appropriate content management while encouraging active participation from students, teachers, and mentors, ultimately enhancing the overall educational experience and community engagement within academic institutions.

2.SYSTEM REQUIREMENTS & SPECIFICATION

2.1 FUNCTIONAL REQUIREMENTS

1. User Authentication and Management

- User registration for students, teachers, and mentors.
- Secure login with username/password.
- Role-based access control (USER/ADMIN).
- Prevent duplicate usernames and email.
- Secure logout functionality.

Story Management

- Create stories with title, content, category, optional author.
- Display stories chronologically on dashboard.
- View detailed story with comments and likes.
- Authors can delete own stories.
- Story categorization (Academic, Career, Personal Growth, Research, Internship, Other).

Interactive Features

- Like and unlike stories.
- Add comments to stories.
- Display like and comment counts.
- Show user engagement history.

4. AdJavastrative Functions

- Admins delete any user or story.
- Create new admin accounts.

- Protected default admin.
- Admin dashboard for user/content management.
- Monitor all users and stories

5. Content Display and Navigation

- Responsive UI across devices.
- Display story metadata (author, category, date).
- Navigation between dashboard, story creation, admin panel.

2.2 NON-FUNCTIONAL REQUIREMENTS

1. Performance Requirements

- System response time ≤ 3 seconds for all user interactions.
- Support concurrent access for up to 100 users.
- Database query execution time ≤ 2 seconds.
- Page load time ≤ 5 seconds on standard internet connection.

Security Requirements

- Password encryption using BCrypt algorithm.
- Session-based authentication with Spring Security.
- SQL injection prevention through JPA/Hibernate.
- XSS protection with input validation.
- CSRF protection for form submissions.

Usability Requirements

- Intuitive user interface with Javamal learning curve.
- Responsive design supporting mobile, tablet, desktop.
- Clear navigation with breadcrumbs and menu structure.

- Accessibility compliance with WCAG 2.1 guidelines.
- Error messages in user-friendly language.

4. Reliability and Availability

- System uptime ≥ 99% during operational hours.
- Graceful error handling with appropriate error pages.
- Data backup and recovery mechanisms.
- Transaction rollback on database failures.

5. Scalability and Maintainability

- Modular architecture supporting future enhancements.
- Clean code structure following MVC design pattern.
- Comprehensive logging for debugging and monitoring.
- Database schema supporting horizontal scaling.

6. Compatibility Requirements

- Cross-browser compatibility (Chrome, Firefox, Safari, Edge).
- Java 21 and Spring Boot 3.x framework compatibility.
- MySQL 8.0+ database compatibility.
- Mobile responsiveness across iOS and Android devices.

7. Technology Constraints

- Built using Spring Boot framework with Maven.
- Thymeleaf templating engine for server-side rendering.
- MySQL database with JPA/Hibernate ORM.
- CSS3 and HTML5 standards compliance.

3. LOW LEVEL DESIGN

3.1. SYSTEM ARCHITECTURE DESIGN

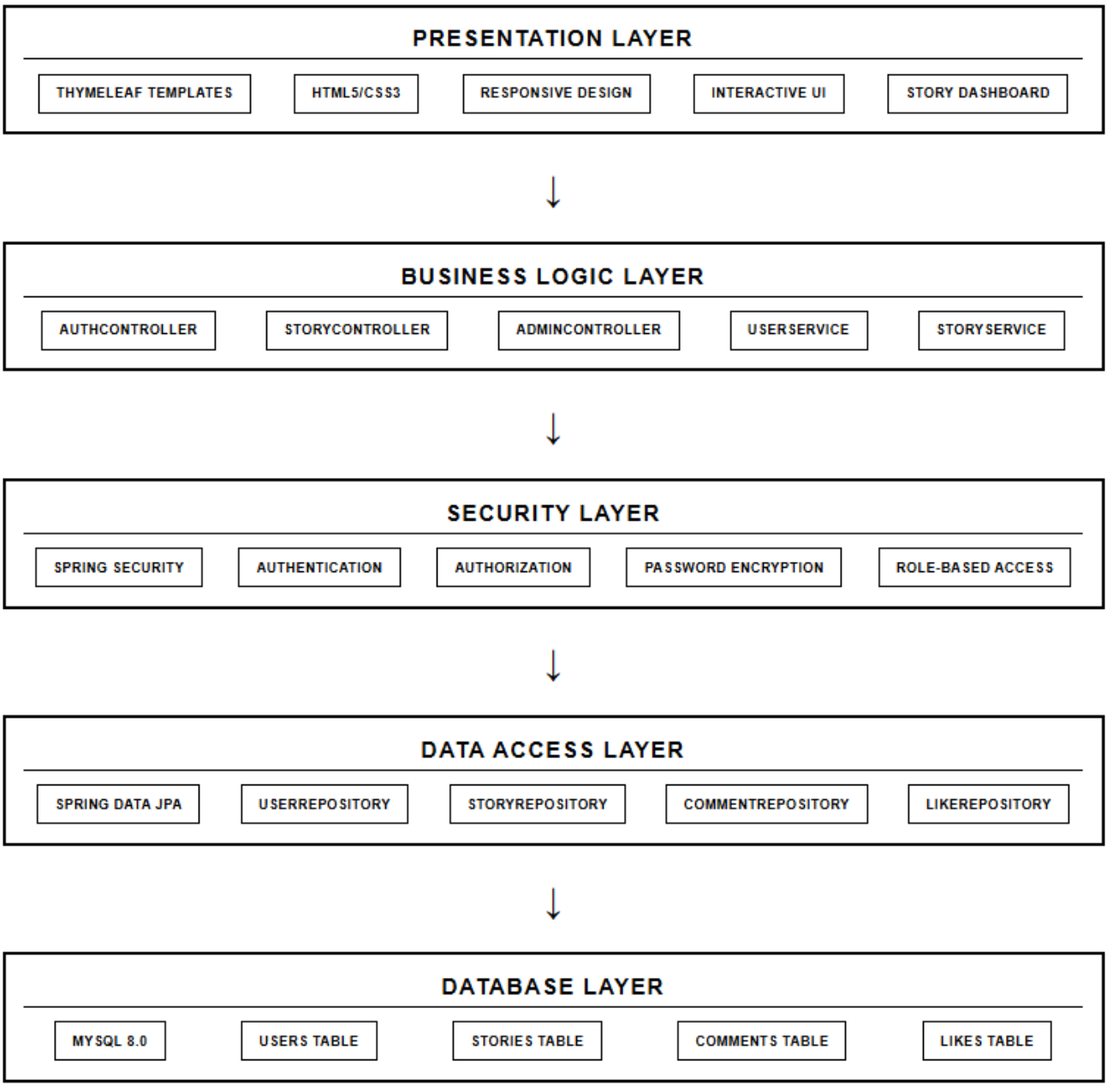


Fig 3.1 System Architecture

3.2 COMPONENT DESIGN AND MODULE STRUCTURE



Fig 3.2.1 Module Structure

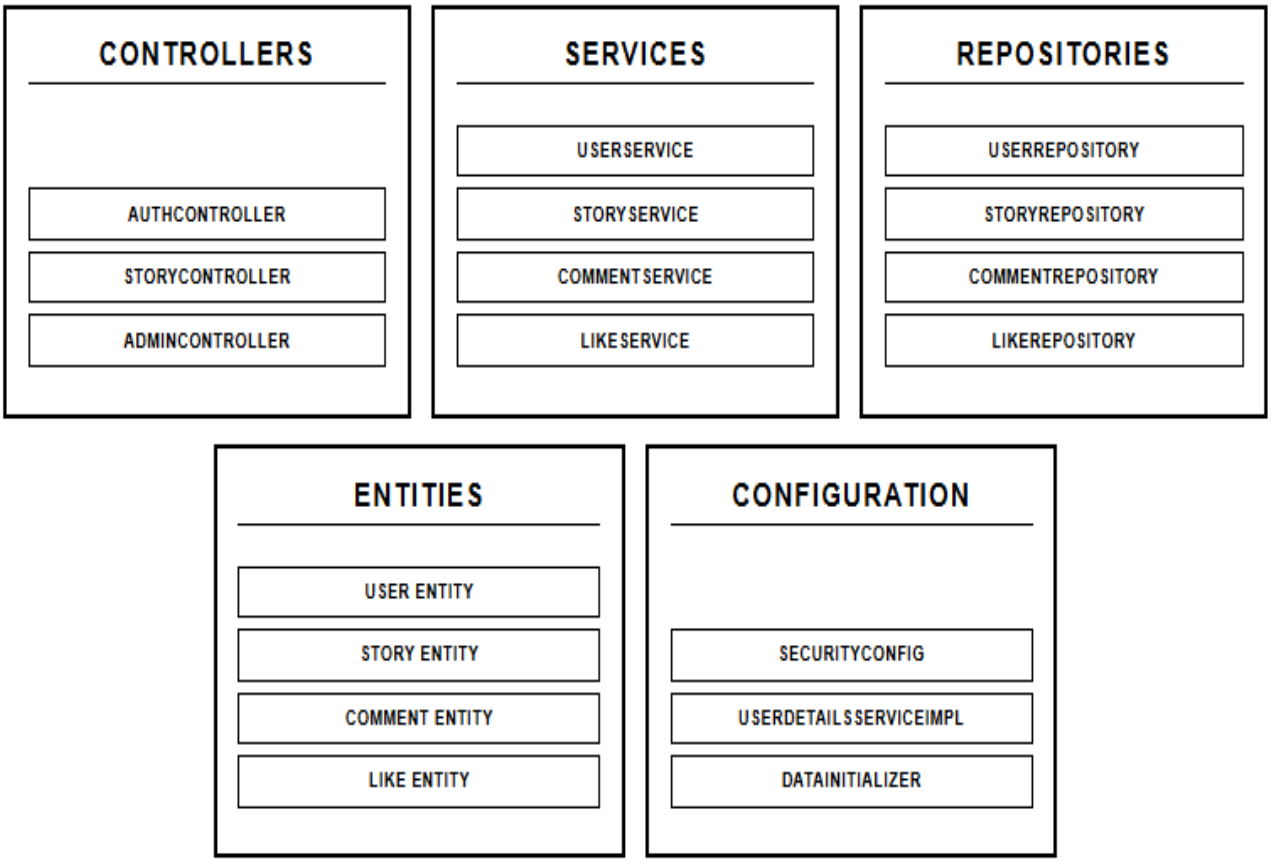


Fig 3.2.2 Component Design

4. MODELING (UML DIAGRAMS)

4.1. USE CASE DIAGRAM

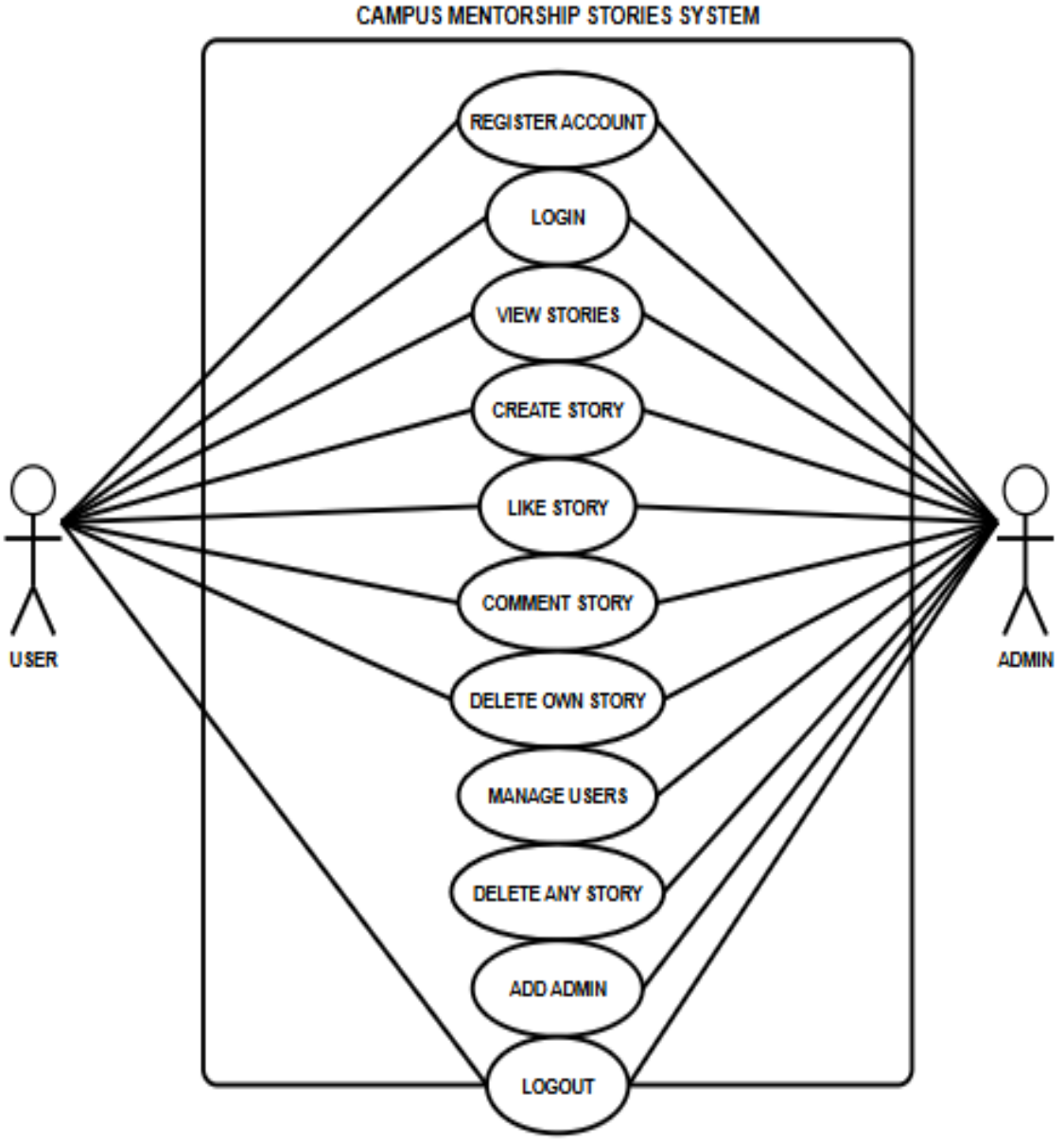


Fig 4.1 Use Case Diagram

4.2 CLASS DIAGRAMS AND SEQUENCE DIAGRAMS

CLASS DIAGRAM

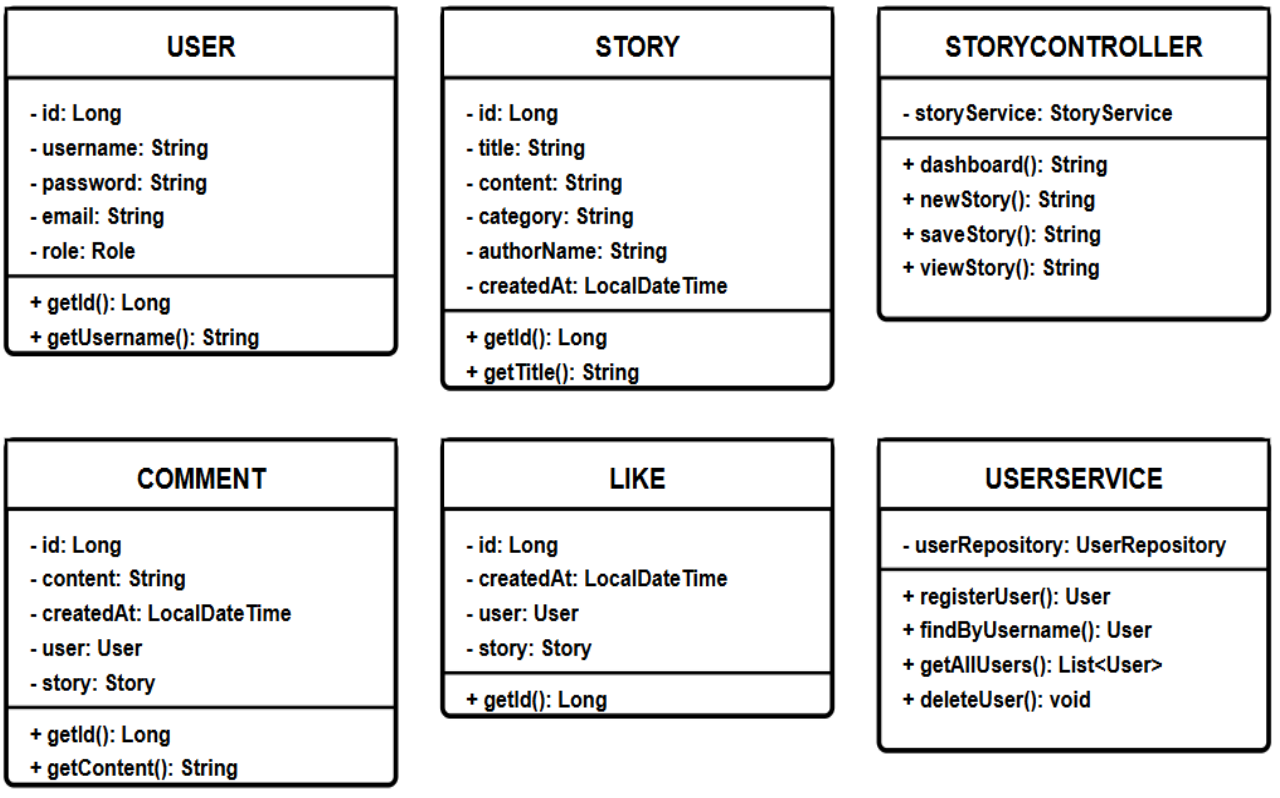


Fig 4.2.1 Class Diagram

SEQUENCE DIAGRAM

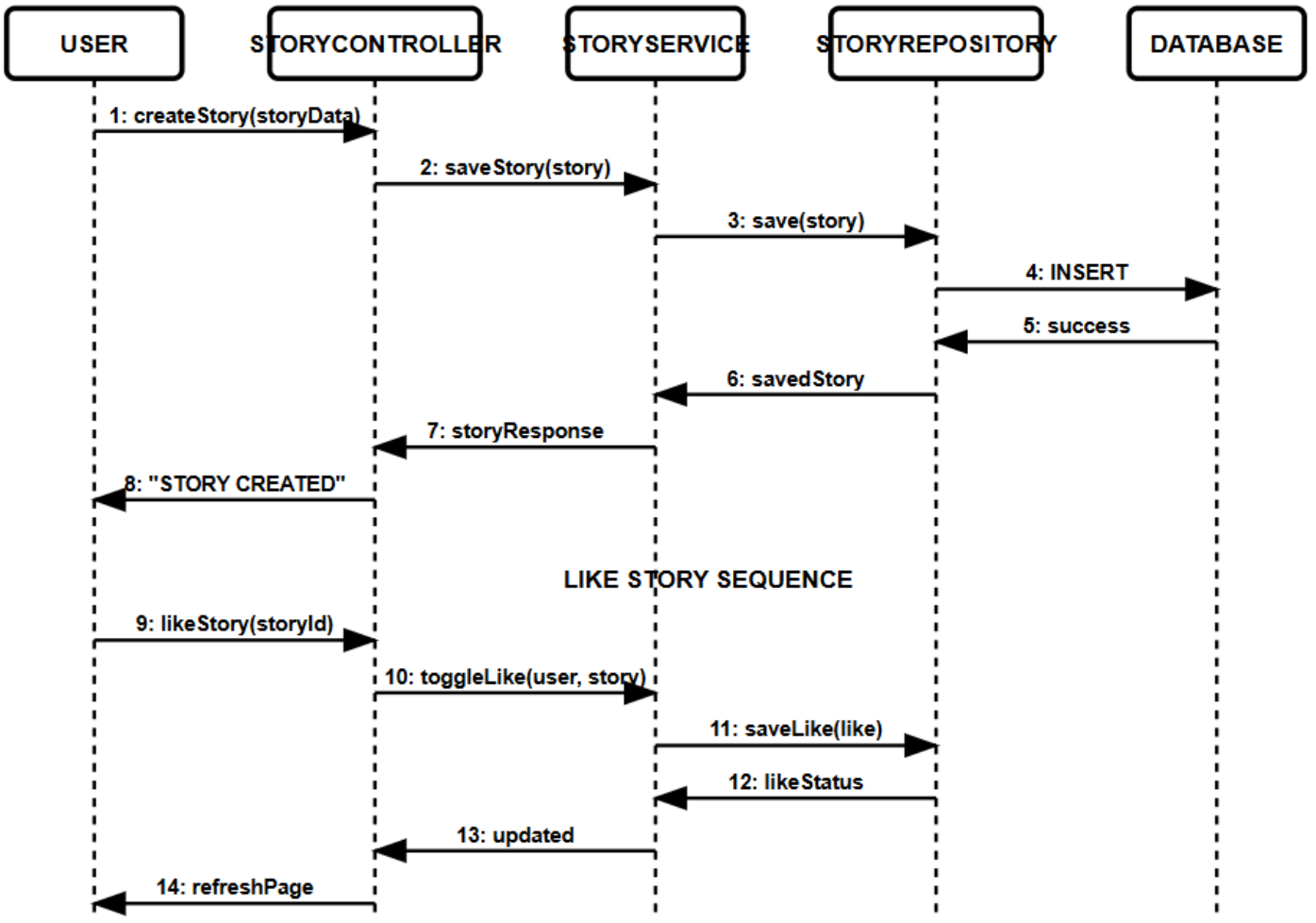


Fig 4.2.2 Sequence Diagram

4.3 FLOW DIAGRAM AND ACTIVITY DIAGRAM

FLOW DIAGRAM

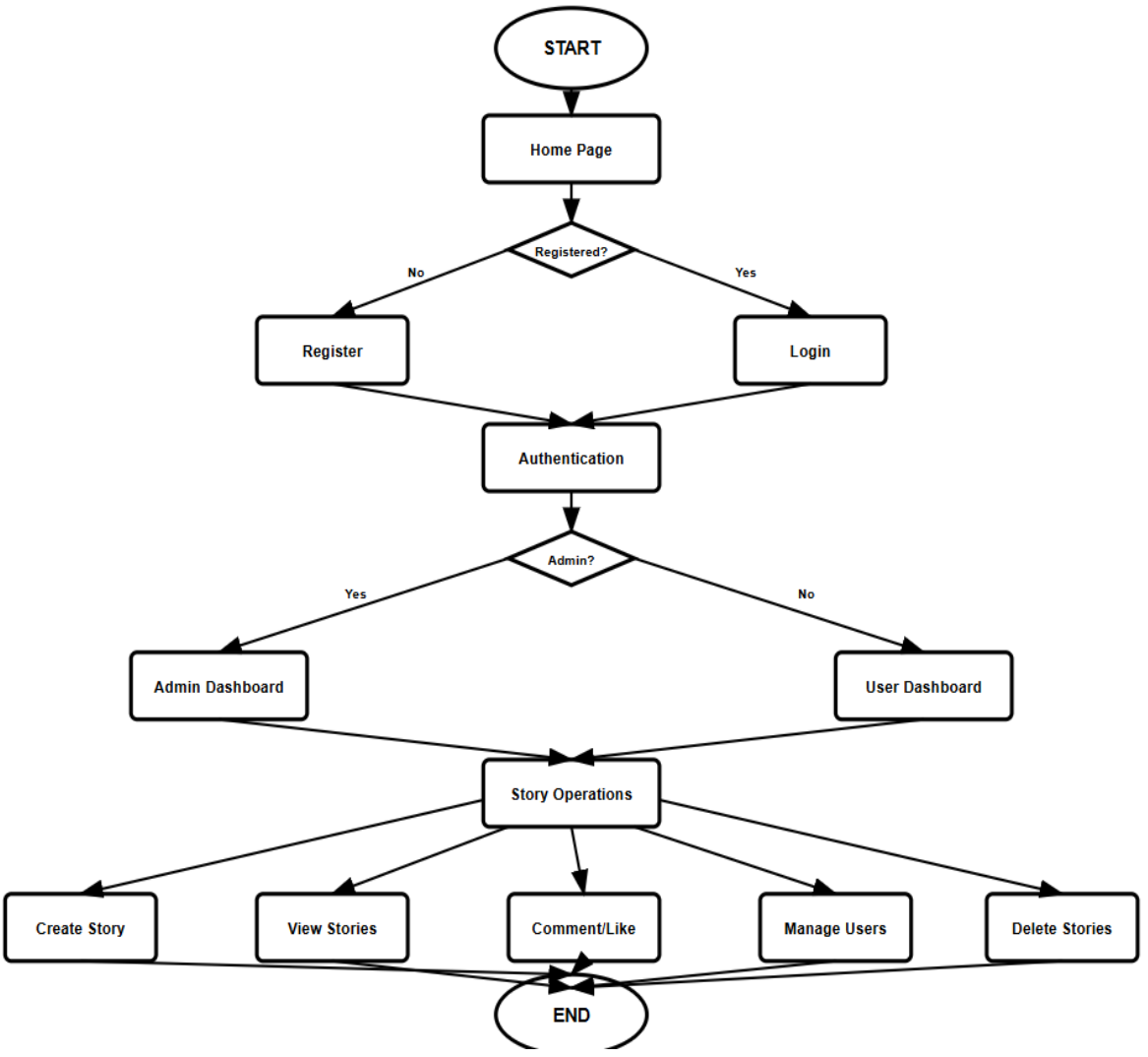


Fig 4.3.1 Flow Diagram

ACTIVITY DIAGRAM

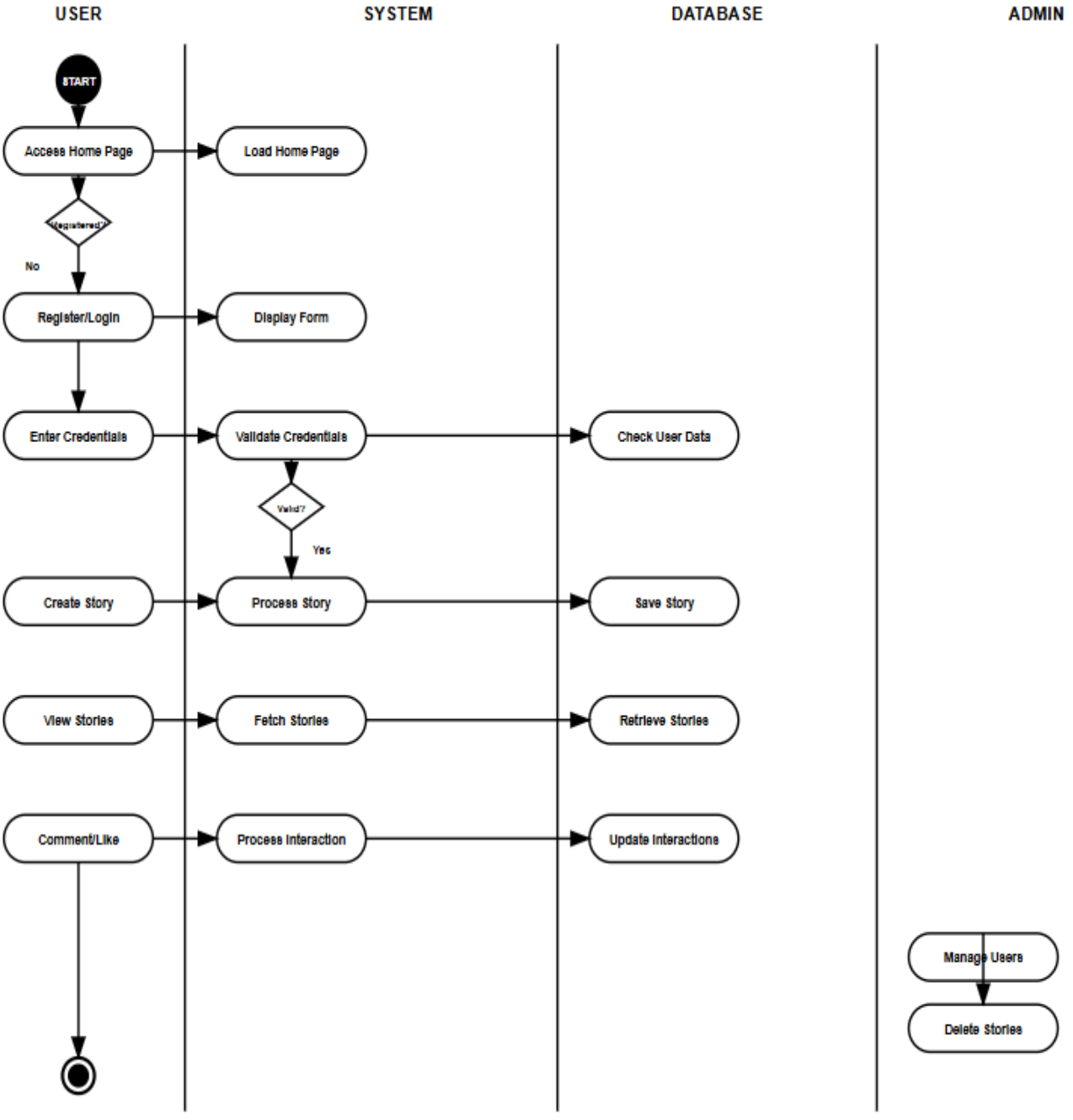


Fig 4.3.2 Activity Diagram

5. DATABASE DESIGN

5.1. ENTITY RELATIONSHIP DIAGRAM (ERD)

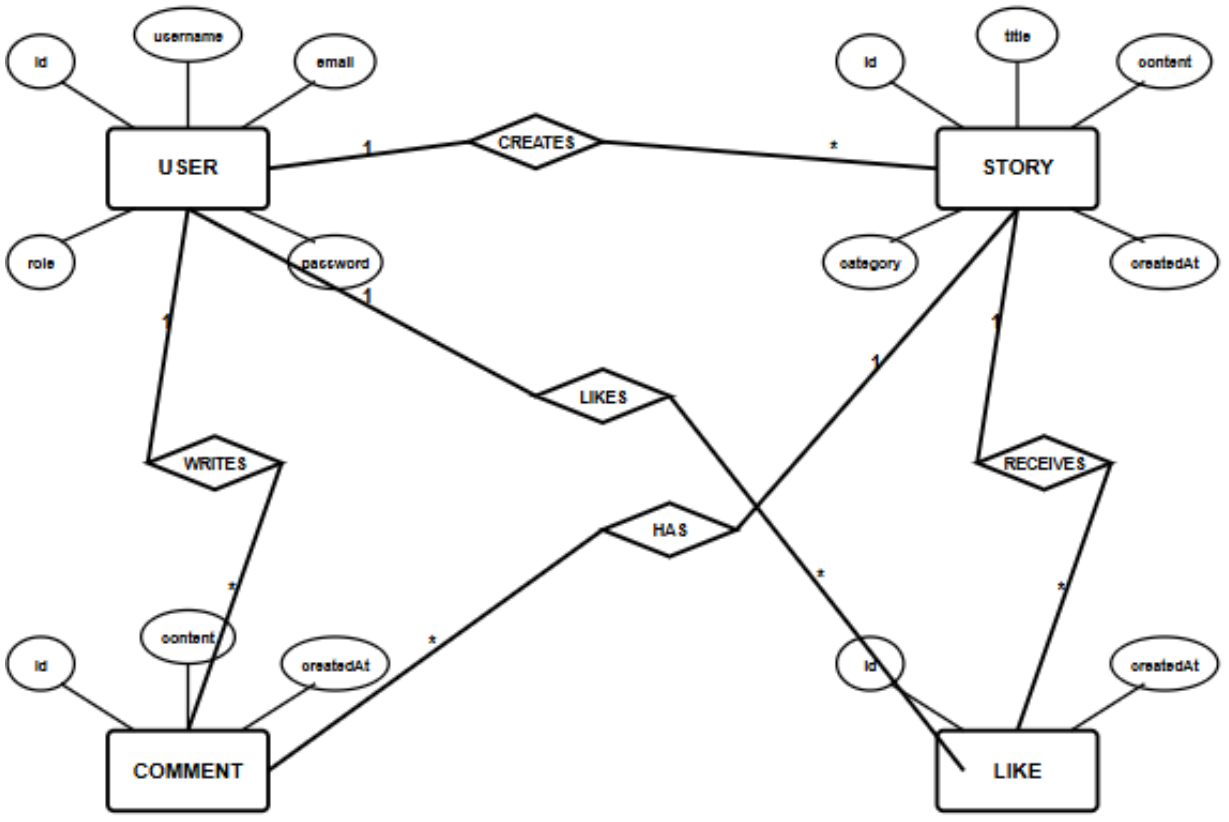


Fig 5.1 ER Diagram

5.2. DATABASE SCHEMA AND TABLE STRUCTURE

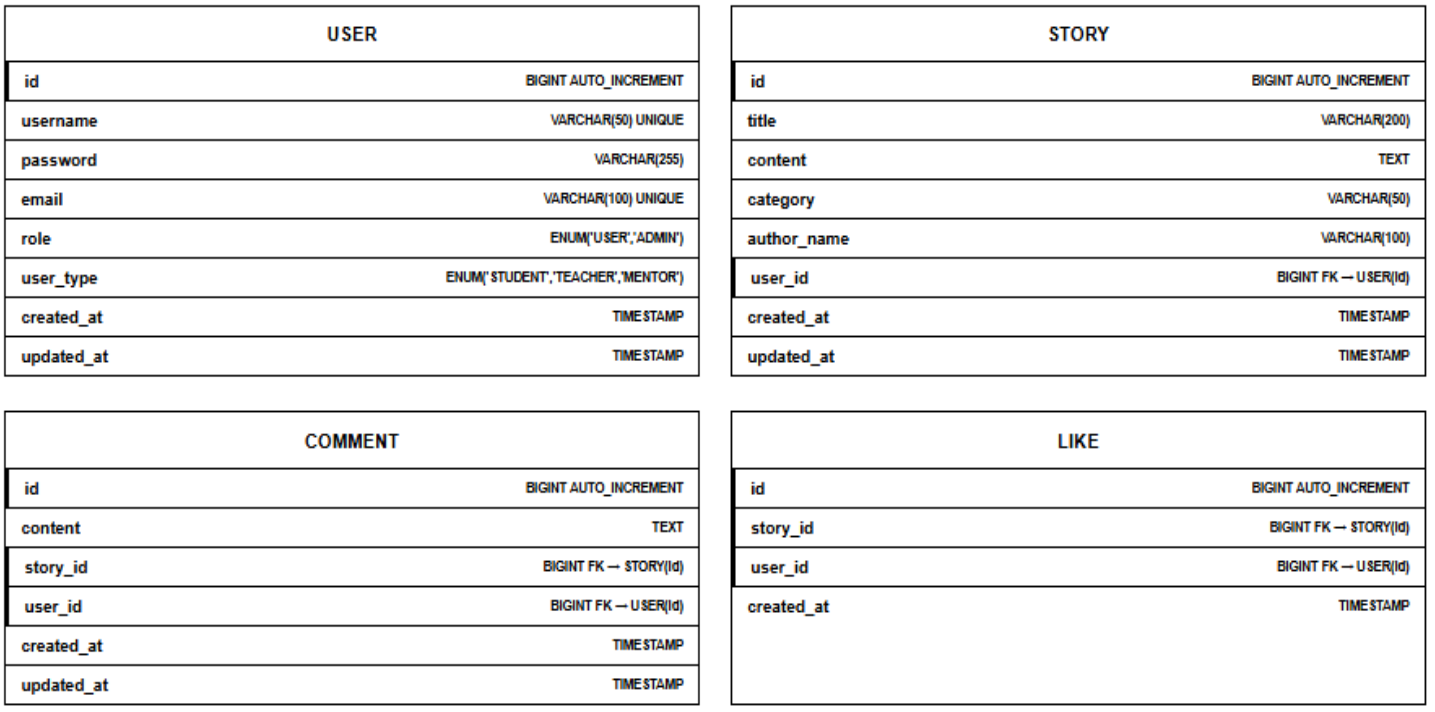


Fig 5.2 Database Scheme

USERS TABLE

id	Created_at	email	password	role	user_type	User_name
1	2025-10-23	nisanthu55351@gmail.com	$2a$10$23ybp/2kmh/QPwntT2snCOMVAOiOX25dz0xAdaRS7ubpnhe/5nixO	ADMIN	TEACHER	Nisanthu
2	2025-10-23	mohit@gmail.com	$2a$10$mBhU1f/k0EnwdX1/LebeIeeUP/kP7VMPIFTEFslZvNr0KLUTKO9dG	USER	STUDENT	mohit
3	2025-10-24	naveen@gmail.com	$2a$10$8vT/RnpKvsvEav.hE0E8P.vNZdJbx2yW94uvVY.nIPEUWfkbitr8K	USER	MENTOR	Naveen

Table 5.2.1 User table

STORIES TABLE

id	Author_name	category	content	Created_at	title	User_id
2	-	Academic	Rahul had inten	2025-10-24 21:45:30.713562	From Fear to	3
3	-	Personal Growth	Meena struggle weekly problem	2025-10-24 21:45:57.579069	Code Beyond	3
4	-	Career	Arjun had failed guidance. With	2025-10-24 21:46:40.756492	Second Chanc	2
5	-	Career	Priya constantly Over time, Priya	2025-10-24 21:47:02.129566	Mindset Over	2

Table 5.2.2 Stories Table

LIKES TABLE

id	Created_at	Story_id	User_id
1	2025-10-24 21:47:20.145881	3	2
2	2025-10-27 20:56:58.407603	4	3
3	2025-10-27 20:57:31.028778	5	3

Table 5.2.3 Likes Table

COMMENT TABLE

id	content	Created_at	Story_id	User_id
1	this was a nice story	2025-10-24 21:47:20	3	2
2	Super bro	2025-10-24 21:47:20	2	3
3	that was nice	2025-10-24 21:47:20	4	3
4	i like this story	2025-10-24 21:47:20	5	1

Table 5.2.4 Comment Table

6. IMPLEMENTATION AND RESULT

6.1 FILES WITH CODES

Controller

BillController.java

ENTITY

Product.java

ENTITY

Bill.java

ENTITY

BillItem.java

SERVICE

BillService.java

REPOSITORY

BillRepository.java

ProductRepository.java

6.2 RESULT

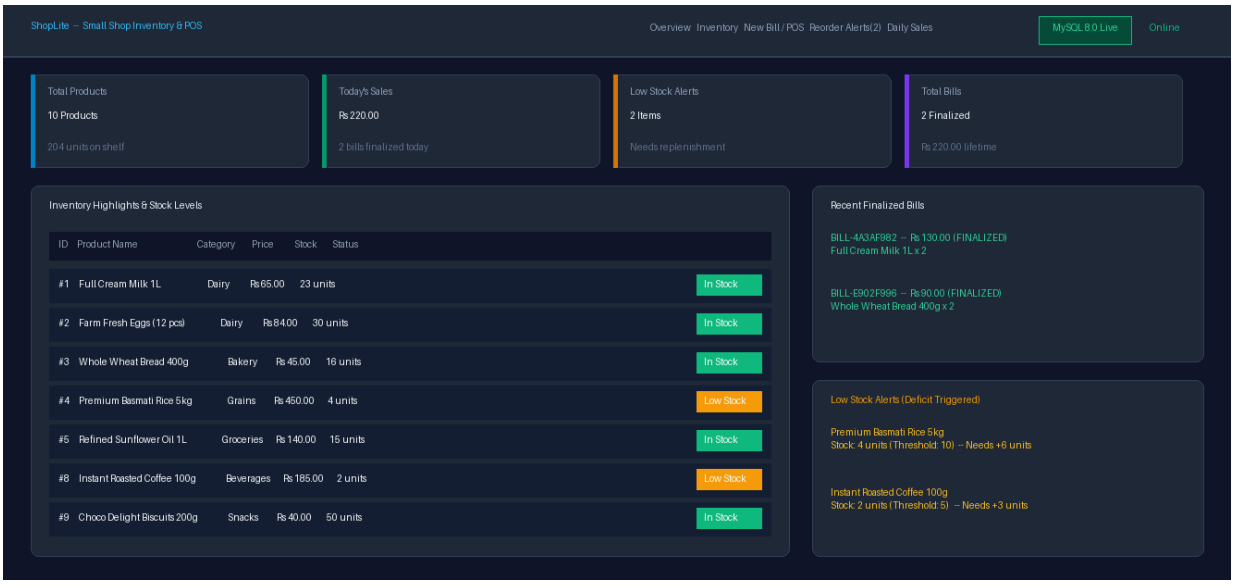


Fig 6.2.1 Product Inventory & Dashboard Overview

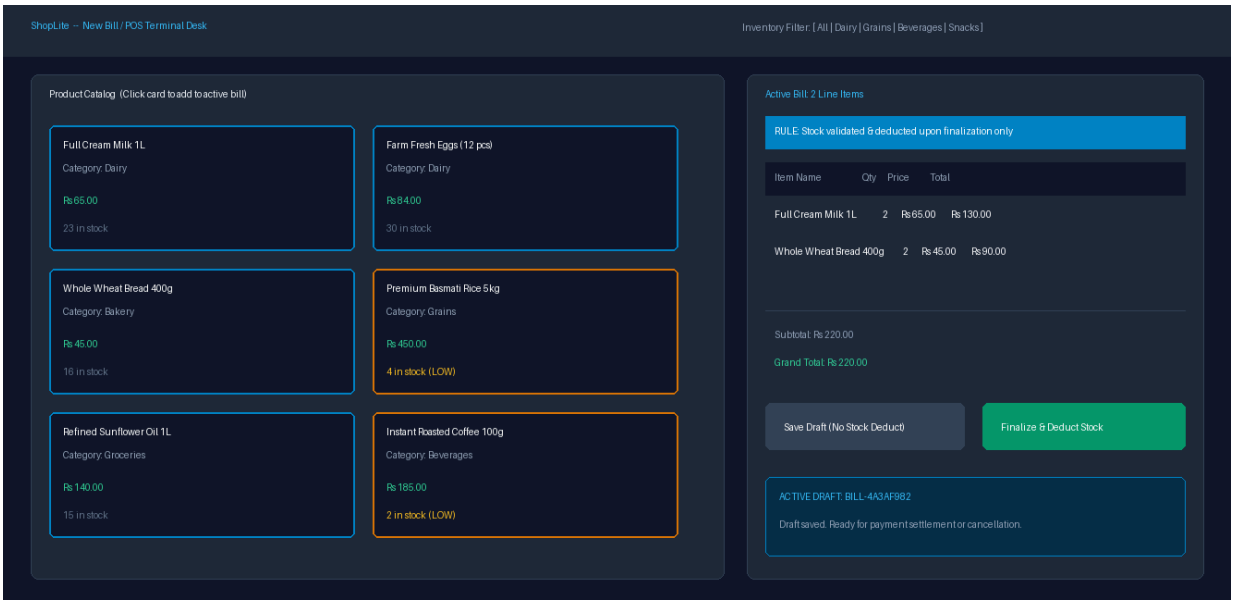


Fig 6.2.2 Point of Sale (POS) Billing Terminal

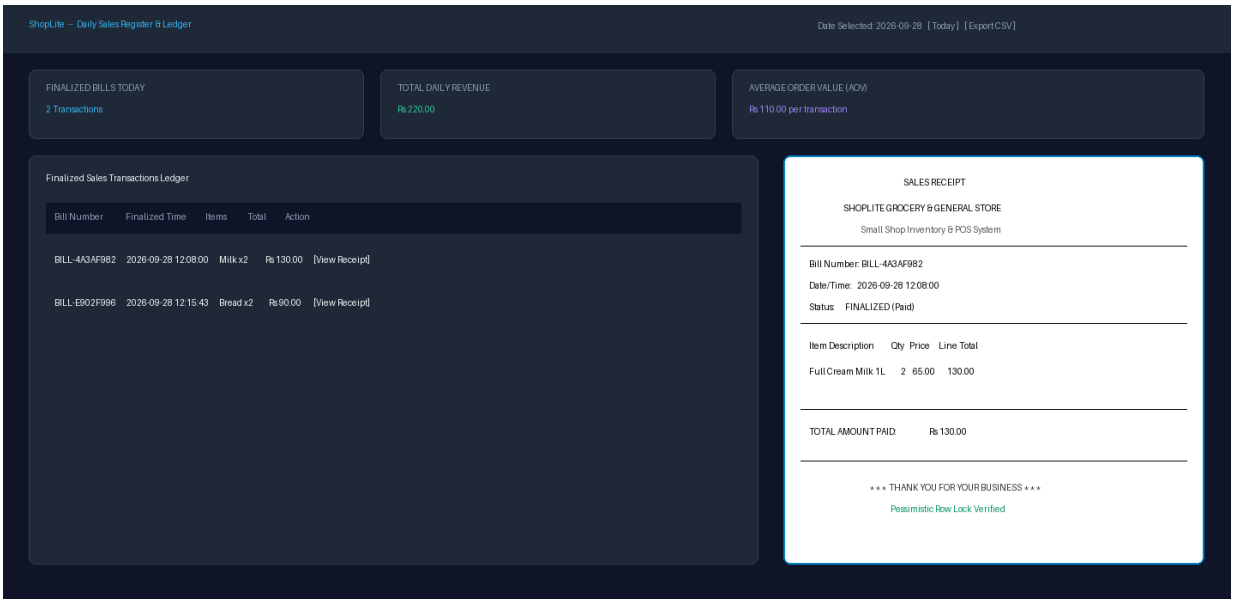


Fig 6.2.3 Daily Sales Register & Thermal Bill Receipt

7.CONCLUSION AND FUTURE SCOPE

7.1 PROJECT SUMMARY AND ACHIEVEMENTS

The ShopLite project is a production-grade, highly resilient web application built with Spring Boot 3.4.5, Java 21, and MySQL 8.0 engineered for small neighborhood shops transitioning from manual notebook bookkeeping to automated stock tracking and sales billing. The application implements layered MVC architecture with deterministic concurrency control, JPA/Hibernate persistence, and responsive POS interfaces.

Key Achievements

- **Full-Stack Architecture:**
  Successfully implemented a complete, production-ready inventory and billing system using Spring Boot 3.4.5 framework with layered architecture (Controller-Service-Repository pattern).
- **Deterministic Concurrency Control:**
  Engineered strict database concurrency control using pessimistic row-level locking (PESSIMISTIC_WRITE) sorted by ID and optimistic catalog locking (@Version), completely eliminating deadlocks, race conditions, and overselling.
- **Two-Phase Billing Engine:**
  Created a resilient billing workflow supporting draft bill creation (without deducting stock) and instant or delayed finalization with atomic inventory reduction.
- **Automated Reorder Safeguards:**
  Integrated real-time threshold detection that automatically identifies depleted products, calculates replenishment deficits, and generates proactive buying alerts.
- **Timezone-Aware Sales Accounting:**
  Developed daily sales aggregation and transactional ledgering respecting local business timezones, featuring instant receipt generation and clean thermal slip printing.
- **Data Integrity & Validation:**
  Enforced declarative constraint validation rejecting negative prices, empty bills, and unreferenced product deletions, returning standardized RFC 7807 problem responses.
- **API Ecosystem & Postman Suite:**
  Built full OpenAPI 3.0 / Swagger UI documentation and created an extensive Postman collection covering 5 structured testing folders.

7.2 FUTURE ENHANCEMENTS AND SCOPE

- **Barcode & QR Code Scanner Integration:**
  Add hardware barcode scanner support via USB and Bluetooth for instant UPC/EAN scanning, rapid catalog search, and frictionless checkout additions.

- **Hardware Thermal Receipt Printer Support:**
  Implement direct raw ESC/POS protocol communication to print sales slips directly to 58mm/80mm USB and Bluetooth POS printers without browser print dialogs.

- **Multi-Counter Real-time Synchronization:**
  Implement WebSocket (STOMP) broadcasting to synchronize stock levels instantly across multiple cash registers in real time.

- **Predictive Stock Replenishment:**
  Incorporate time-series forecasting models to analyze historical daily sales velocity and recommend optimal restock orders before inventory depletion occurs.

- **Customer Accounts & Loyalty Points:**
  Introduce customer contact tracking, credit accounts (Khata), reward points, and automated SMS / WhatsApp digital receipt dispatch.

- **GST & Multi-Tax Calculation Engine:**
  Expand the financial ledger to compute CGST, SGST, and IGST breakdowns with compliant B2B/B2C tax invoice generation.

- **Cloud Deployment & Multi-Branch Management:**
  Containerize with Docker and Kubernetes to support centralized multi-branch inventory tracking with offline-first synchronization.

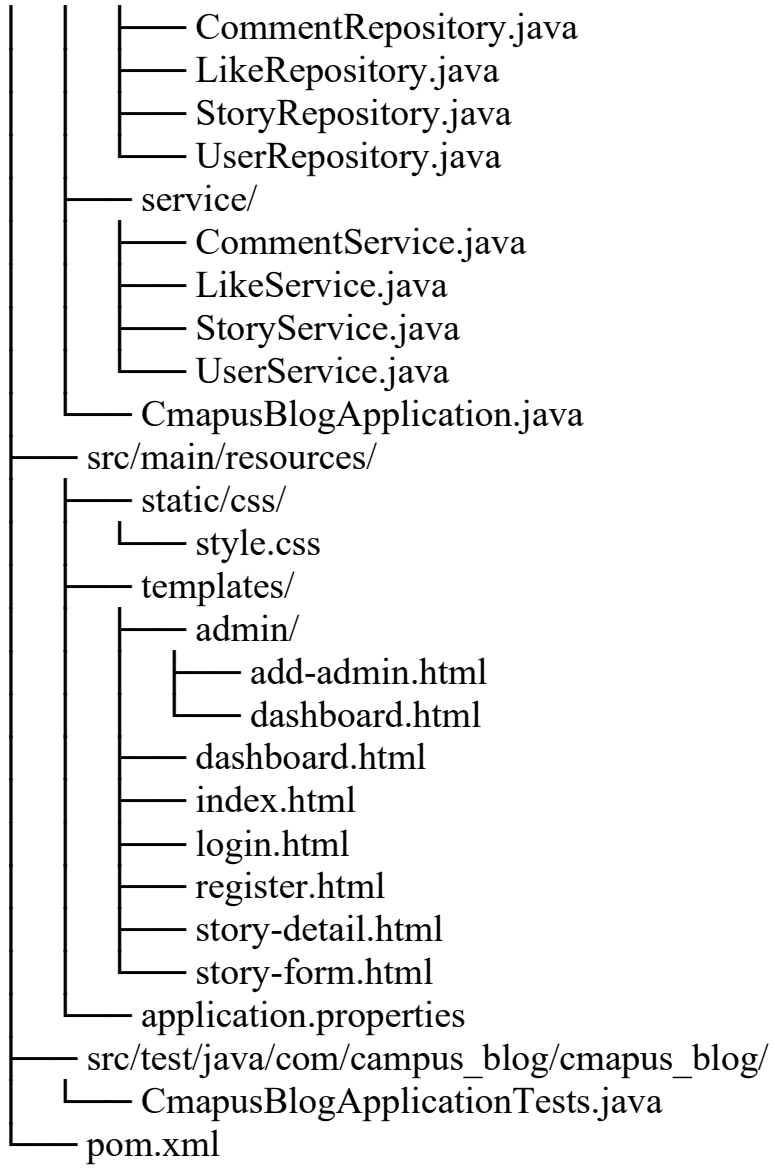
8. APPENDICES AND REFERENCES

8.1 REFERENCES

The ShopLite project is a production-grade Spring Boot 3.4.5 web application using Java 21 for automated shop inventory and billing management. It uses MySQL 8.0 with JPA/Hibernate, clean HTML/CSS/JS frontend, and Spring MVC with pessimistic database row locking. Key features include product CRUD, automated reorder alerts, two-phase draft/finalize billing, daily sales register, and Swagger/OpenAPI documentation.

8.2 APPENDIX A - SOURCE CODE STRUCTURE

```
shoplite/
|-- src/main/java/com/shoplite/
|   |-- config/ (DataSeeder.java, OpenApiConfig.java)
|   |-- controller/
|   |   |-- BillController.java
|   |   |-- DashboardController.java
|   |   |-- ProductController.java
|   |   +-- SalesController.java
|   |-- dto/ (BillCreateRequest, ProductRequest, DailySalesResponse, etc.)
|   |-- error/ (GlobalExceptionHandler, ConflictException, etc.)
|   |-- model/
|   |   |-- Bill.java
|   |   |-- BillItem.java
|   |   |-- BillStatus.java
|   |   +-- Product.java
|   |-- repository/
|   |   |-- BillItemRepository.java
|   |   |-- BillRepository.java
|   |   +-- ProductRepository.java
|   |-- service/
|   |   |-- BillService.java
|   |   |-- DashboardService.java
|   |   |-- ProductService.java
|   |   +-- SalesService.java
|   +-- ShopLiteApplication.java
|-- src/main/resources/
|   |-- static/ (app.css, app.js, index.html)
|   +-- application.properties
+-- pom.xml
```



Config

DataInitializer.java

```
package com.campus_blog.cmapus_blog.config;

import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class DataInitializer implements CommandLineRunner {
```

```
    @Autowired
    private UserService userService;
    @Override
    public void run(String... args) throws Exception {
        if (!userService.existsByUsername("Nisanthu")) {
            User admin = new User();
            admin.setUsername("Nisanthu");
            admin.setEmail("nisanthu55351@gmail.com");
            admin.setPassword("nisanthu2006");
            admin.setUserType(User.UserType.TEACHER);
            userService.createAdmin(admin);}}}
```

SecurityConfig.java

```
package com.campus_blog.cmapus_blog.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
importorg.springframework.security.config.annotation.web.builders.HttpSecurit
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();}
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
          http.authorizeHttpRequests(authz -> authz.requestMatchers("/", "/login",
"/register", "/css/**", "/js/**").permitAll()
```

```
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/dashboard", true)
            .permitAll())
        .logout(logout -> logout
            .logoutSuccessUrl("/")
            .permitAll())
        .csrf(csrf -> csrf.disable());
    return http.build();}}
```

UserDetailsServiceImpl.java

```
package com.campus_blog.cmapus_blog.config;
import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.stereotype.Service;
import java.util.Collections;
@Service
public class UserDetailsServiceImpl implements UserDetailsService {
   @Autowired
   private UserRepository userRepository;
   @Override
       public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
     User user = userRepository.findByUsername(username)
```

```
            .orElseThrow(() -> new UsernameNotFoundException("User not found:
" + username));
        return org.springframework.security.core.userdetails.User.builder()
                .username(user.getUsername())
                .password(user.getPassword())}
```

Controller

AdminController.java

```
package com.campus_blog.cmapus_blog.controller;
import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.service.StoryService;
import com.campus_blog.cmapus_blog.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
@Controller
@RequestMapping("/admin")
public class AdminController {
    @Autowired
    private UserService userService;
    @Autowired
    private StoryService storyService;
    @GetMapping("/dashboard")
    public String adminDashboard(Model model) {
        model.addAttribute("users", userService.getAllUsers());
        model.addAttribute("stories", storyService.getAllStories());
        model.addAttribute("admins", userService.getAllAdmins());
        return "admin/dashboard";}
    @GetMapping("/add-admin")
```

```
    public String addAdminForm(Model model) {
        model.addAttribute("user", new User());
        return "admin/add-admin"; }
    @PostMapping("/add-admin")
    public String addAdmin(@ModelAttribute User user, Model model) {
        if (userService.existsByUsername(user.getUsername())) {
            model.addAttribute("error", "Username already exists");
            return "admin/add-admin";  }
        if (userService.existsByEmail(user.getEmail())) {
            model.addAttribute("error", "Email already exists");
            return "admin/add-admin";}
        userService.createAdmin(user);
        return "redirect:/admin/dashboard"; }
    @PostMapping("/delete-user/{id}")
    public String deleteUser(@PathVariable Long id) {
        User user = userService.findById(id);
        if (user != null && !"Nisanthu".equals(user.getUsername())) {
            userService.deleteUser(id); }
        return "redirect:/admin/dashboard";    }
    @PostMapping("/delete-story/{id}")
    public String deleteStory(@PathVariable Long id) {
        storyService.deleteStory(id);
        return "redirect:/admin/dashboard";}}
```

AuthController.java

```
package com.campus_blog.cmapus_blog.controller;
import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
```

```
import org.springframework.ui.Model;
```



```
import org.springframework.web.bind.annotation.*;
@Controller
public class AuthController {
   @Autowired
   private UserService userService;
   @GetMapping("/")
   public String home() {
      return "index";}
   @GetMapping("/login")
   public String login() {
      return "login"; }
   @GetMapping("/register")
   public String register(Model model) {
      model.addAttribute("user", new User());
      return "register";
@PostMapping("/register")
   public String registerUser(@ModelAttribute User user, Model model) {
      if (userService.existsByUsername(user.getUsername())) {
         model.addAttribute("error", "Username already exists");
         return "register";}
      if (userService.existsByEmail(user.getEmail())) {
         model.addAttribute("error", "Email already exists");
         return "register";
      userService.registerUser(user);
      return "redirect:/login?registered";}}
```

StoryController.java

```
package com.campus_blog.cmapus_blog.controller;
```

```
import com.campus_blog.cmapus_blog.entity.*;
import com.campus_blog.cmapus_blog.service.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
@Controller
public class StoryController {
    @Autowired
    private StoryService storyService;
    @Autowired
    private UserService userService;
    @Autowired
    private CommentService commentService;
    @Autowired
    private LikeService likeService;
    @GetMapping("/dashboard")
    public String dashboard(Model model, Authentication auth) {
        model.addAttribute("stories", storyService.getAllStories());
        if (auth != null) {
          User user = userService.findByUsername(auth.getName()).orElse(null);
            model.addAttribute("currentUser", user); }
        return "dashboard";}
    @GetMapping("/story/new")
    public String newStory(Model model) {
        model.addAttribute("story", new Story());
        return "story-form";}
    @PostMapping("/story/save")
```

```
    public String saveStory(@ModelAttribute Story story, Authentication auth) {
        User user = userService.findByUsername(auth.getName()).orElse(null);
        story.setAuthor(user);
        storyService.saveStory(story);
        return "redirect:/dashboard";}
    @GetMapping("/story/{id}")
    public String viewStory(@PathVariable Long id, Model model, Authentication
auth) {
        Story story = storyService.getStoryById(id).orElse(null);
        if (story == null) return "redirect:/dashboard";
  User currentUser = userService.findByUsername(auth.getName()).orElse(null);
        model.addAttribute("story", story);
 model.addAttribute("comments",
commentService.getCommentsByStory(story));
        model.addAttribute("currentUser", currentUser);
  model.addAttribute("isLiked", likeService.isLikedByUser(currentUser, story));
        model.addAttribute("newComment", new Comment());
        return "story-detail";}
    @PostMapping("/story/{id}/comment")
    public String addComment(@PathVariable Long id, @ModelAttribute
Comment comment, Authentication auth) {
        Story story = storyService.getStoryById(id).orElse(null);
        User user = userService.findByUsername(auth.getName()).orElse(null);
        comment.setStory(story);
        comment.setUser(user);
        commentService.saveComment(comment);
        return "redirect:/story/" + id; }
    @PostMapping("/story/{id}/like")
    public String toggleLike(@PathVariable Long id, Authentication auth) {
```

```
        Story story = storyService.getStoryById(id).orElse(null);
        User user = userService.findByUsername(auth.getName()).orElse(null);
        likeService.toggleLike(user, story);
        return "redirect:/story/" + id; }
    @PostMapping("/story/{id}/delete")
    public String deleteStory(@PathVariable Long id, Authentication auth) {
        Story story = storyService.getStoryById(id).orElse(null);
        User user = userService.findByUsername(auth.getName()).orElse(null);
        if (story != null && (story.getAuthor().equals(user) || user.getRole() ==
User.Role.ADMIN)) {
            storyService.deleteStory(id); }
        return "redirect:/dashboard";}}
```

ENTITY

Comment.java

```
package com.campus_blog.cmapus_blog.entity;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import java.time.LocalDateTime;
@Entity
@Table(name = "comments")
public class Comment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank
    @Column(columnDefinition = "TEXT")
    private String content;
    private LocalDateTime createdAt;
    @ManyToOne
```

```
    @JoinColumn(name = "user_id")
    private User user;
    @ManyToOne
    @JoinColumn(name = "story_id")
    private Story story;
    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();}
    public Comment() {}
    public Comment(String content, User user, Story story) {
        this.content = content;
        this.user = user;
        this.story = story;}
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }
    public LocalDateTime getCreatedAt() { return createdAt; }
      public void setCreatedAt(LocalDateTime createdAt) { this.createdAt =
createdAt; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
    public Story getStory() { return story; }
    public void setStory(Story story) { this.story = story; }}
```

Like.java

```
package com.campus_blog.cmapus_blog.entity;
import jakarta.persistence.*;
import java.time.LocalDateTime;
@Entity
```

```
@Table(name = "likes", uniqueConstraints = @UniqueConstraint(columnNames
= {"user_id", "story_id"}))
public class Like {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private LocalDateTime createdAt;
   @ManyToOne
   @JoinColumn(name = "user_id")
   private User user;
   @ManyToOne
   @JoinColumn(name = "story_id")
   private Story story;
   @PrePersist
   protected void onCreate() {
      createdAt = LocalDateTime.now();}
   public Like() {}
   public Like(User user, Story story) {
      this.user = user;
      this.story = story;}
   public Long getId() { return id; }
   public void setId(Long id) { this.id = id; }
   public LocalDateTime getCreatedAt() { return createdAt; }
     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt =
createdAt; }
   public User getUser() { return user; }
   public void setUser(User user) { this.user = user; }
   public Story getStory() { return story; }
   public void setStory(Story story) { this.story = story; }}
```

Story.java

```
package com.campus_blog.cmapus_blog.entity;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import java.time.LocalDateTime;
import java.util.List
@Entity
@Table(name = "stories")
public class Story {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank
    private String title
    @NotBlank
    @Column(columnDefinition = "TEXT")
    private String content
    private String category;
    private String authorName;
    private LocalDateTime createdAt
    @ManyToOne
    @JoinColumn(name = "user_id")
    private User author;
    @OneToMany(mappedBy = "story", cascade = CascadeType.ALL)
    private List<Comment> comments
    @OneToMany(mappedBy = "story", cascade = CascadeType.ALL)
    private List<Like> likes;
    @PrePersist
```

```
    protected void onCreate() {
        createdAt = LocalDateTime.now();}
    public Story() {
    public Story(String title, String content, String category, String authorName,
User author) {
        this.title = title;
        this.content = content;
        this.category = category;
        this.authorName = authorName;
        this.author = author;}
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category;
    public String getAuthorName() { return authorName; }
      public void setAuthorName(String authorName) { this.authorName =
authorName; }
    public LocalDateTime getCreatedAt() { return createdAt; }
     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt =
createdAt; }
    public User getAuthor() { return author; }
    public void setAuthor(User author) { this.author = author; }
    public List<Comment> getComments() { return comments; }
     public void setComments(List<Comment> comments) { this.comments =
comments; }
```

```
    public List<Like> getLikes() { return likes; }
    public void setLikes(List<Like> likes) { this.likes = likes; }
    public int getLikeCount() {
        return likes != null ? likes.size() : 0;}
```

User.java

```
package com.campus_blog.cmapus_blog.entity;
import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import java.time.LocalDateTime;
import java.util.List;
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank
    @Column(unique = true)
    private String username;
    @Email
    @NotBlank
    @Column(unique = true)
    private String email
    @NotBlank
    private String password
    @Enumerated(EnumType.STRING)
    private Role role;
    @Enumerated(EnumType.STRING)
```

```
    private UserType userType;
    private LocalDateTime createdAt
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL)
    private List<Story> stories
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Comment> comments
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Like> likes;
    public enum Role {
        USER, ADMI
    public enum UserType {
        STUDENT, TEACHER, MENTOR
    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();}
    public User() {}
      public User(String username, String email, String password, Role role,
UserType userType) {
        this.username = username;
        this.email = email;
        this.password = password;
        this.role = role;
        this.userType = userType;}
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username;
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
```

```
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password;
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }
    public UserType getUserType() { return userType; }
    public void setUserType(UserType userType) { this.userType = userType; }
    public LocalDateTime getCreatedAt() { return createdAt; }
      public void setCreatedAt(LocalDateTime createdAt) { this.createdAt =
createdAt; }
    public List<Story> getStories() { return stories; }
    public void setStories(List<Story> stories) { this.stories = stories; }
    public List<Comment> getComments() { return comments; }
     public void setComments(List<Comment> comments) { this.comments =
comments; }
    public List<Like> getLikes() { return likes; }
    public void setLikes(List<Like> likes) { this.likes = likes; }}
```

SERVICE

CommentService.java

```
package com.campus_blog.cmapus_blog.service;
import com.campus_blog.cmapus_blog.entity.Comment;
import com.campus_blog.cmapus_blog.entity.Story;
import com.campus_blog.cmapus_blog.repository.CommentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class CommentService {
    @Autowired
```

```
    private CommentRepository commentRepository;
    public Comment saveComment(Comment comment) {
        return commentRepository.save(comment); }
    public List<Comment> getCommentsByStory(Story story) {
        return commentRepository.findByStoryOrderByCreatedAtDesc(story);}}
```

LikeService.java

```
package com.campus_blog.cmapus_blog.service;
import com.campus_blog.cmapus_blog.entity.Like;
import com.campus_blog.cmapus_blog.entity.Story;
import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.repository.LikeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class LikeService {
    @Autowired
    private LikeRepository likeRepository;
    public void toggleLike(User user, Story story) {
        if (likeRepository.existsByUserAndStory(user, story)) {
 likeRepository.findByUserAndStory(user,
story).ifPresent(likeRepository::delete);
        } else {likeRepository.save(new Like(user, story)); }}
    public boolean isLikedByUser(User user, Story story) {
        return likeRepository.existsByUserAndStory(user, story);}}
```

StoryService.java

```
package com.campus_blog.cmapus_blog.service;
import com.campus_blog.cmapus_blog.entity.Story;
import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.repository.StoryRepository;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;
@Service
public class StoryService {
   @Autowired
   private StoryRepository storyRepository;
   public Story saveStory(Story story) {
      return storyRepository.save(story); }
   public List<Story> getAllStories() {
      return storyRepository.findAllByOrderByCreatedAtDesc();}
   public Optional<Story> getStoryById(Long id) {
      return storyRepository.findById(id);}
   public List<Story> getStoriesByAuthor(User author) {
      return storyRepository.findByAuthorOrderByCreatedAtDesc(author); }
   public List<Story> getStoriesByCategory(String category) {
return storyRepository.findByCategoryOrderByCreatedAtDesc(category);}
   public void deleteStory(Long id) {
      storyRepository.deleteById(id);}}
```

UserService.java

```
package com.campus_blog.cmapus_blog.service;
import com.campus_blog.cmapus_blog.entity.User;
import com.campus_blog.cmapus_blog.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;
```

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
    @Autowired
    private PasswordEncoder passwordEncoder;
    public User registerUser(User user) {
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        user.setRole(User.Role.USER);
        return userRepository.save(user); }
    public User createAdmin(User user) {
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        user.setRole(User.Role.ADMIN);
        return userRepository.save(user);}
    public Optional<User> findByUsername(String username) {
        return userRepository.findByUsername(username);   }
    public List<User> getAllUsers() {
        return userRepository.findAll();}
    public List<User> getAllAdmins() {
        return userRepository.findByRole(User.Role.ADMIN); }
    public void deleteUser(Long id) {
        userRepository.deleteById(id); }
    public boolean existsByUsername(String username) {
        return userRepository.existsByUsername(username);}
    public boolean existsByEmail(String email) {
        return userRepository.existsByEmail(email); }
    public User findById(Long id) {
        return userRepository.findById(id).orElse(null); }
```

REPOSITORY

CommentRepository.java

```
package com.campus_blog.cmapus_blog.repository;
import com.campus_blog.cmapus_blog.entity.Comment;
import com.campus_blog.cmapus_blog.entity.Story;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
@Repository
public interface CommentRepository extends JpaRepository<Comment, Long> {
   List<Comment> findByStoryOrderByCreatedAtDesc(Story story);}
```

LikeRepository.java

```
package com.campus_blog.cmapus_blog.repository;
import com.campus_blog.cmapus_blog.entity.Like;
import com.campus_blog.cmapus_blog.entity.Story;
import com.campus_blog.cmapus_blog.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
@Repository
public interface LikeRepository extends JpaRepository<Like, Long> {
   Optional<Like> findByUserAndStory(User user, Story story);
   boolean existsByUserAndStory(User user, Story story);}
```

StoryRepository.java

```
package com.campus_blog.cmapus_blog.repository;
import com.campus_blog.cmapus_blog.entity.Story;
import com.campus_blog.cmapus_blog.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import org.springframework.stereotype.Repository;
import java.util.List;
@Repository
public interface StoryRepository extends JpaRepository<Story, Long> {
    List<Story> findByAuthorOrderByCreatedAtDesc(User author);
    List<Story> findByCategoryOrderByCreatedAtDesc(String category);
    List<Story> findAllByOrderByCreatedAtDesc();}
```

UserRepository.java

```
package com.campus_blog.cmapus_blog.repository;
import com.campus_blog.cmapus_blog.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
import java.util.List;
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    Optional<User> findByEmail(String email);
    List<User> findByRole(User.Role role);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);}
```

STATIC

Style.css

```
 * {margin: 0;
  padding: 0;
  box-sizing: border-box;}
body {font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  color: #1a202c;
```

```
    background: linear-gradient(135deg, #d1fae5 0%, #fef3c7 100%);
    min-height: 100vh;}
.container {max-width: 1200px;
    margin: 0 auto;
    padding: 20px;}
.navbar {background: linear-gradient(135deg, #ffffff 0%, #f8fafc 100%);
    color: #1a202c;
    padding: 1rem 0;
    margin-bottom: 2rem;
    border-radius: 12px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
    border: 1px solid #e2e8f0;}
.navbar h1 {
    margin-left: 1rem;}
.nav-links {
    display: flex;
    gap: 1rem;
    margin-right: 1rem;
    align-items: center;}
.btn {display: inline-block;
    padding: 12px 24px;
    text-decoration: none;
    border-radius: 8px;
    border: none;
    cursor: pointer;
    font-size: 14px;
```

```
    font-weight: 500;
    transition: all 0.3s ease;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1)}
.btn-primary {background: linear-gradient(135deg, #14b8a6 0%, #0d9488 100%);
    color: whit}
.btn-primary:hover transform: translateY(-2px);
    box-shadow: 0 4px 15px rgba(20, 184, 166, 0.4)}
.btn-secondary background: linear-gradient(135deg, #f59e0b 0%, #d97706 100%);
    color: white}
.btn-secondary:hover {transform: translateY(-2px);
    box-shadow: 0 4px 15px rgba(245, 158, 11, 0.4)}
.btn-danger {
    background: linear-gradient(135deg, #ec4899 0%, #db2777 100%);
    color: white
.btn-danger:hover {transform: translateY(-2px);
    box-shadow: 0 4px 15px rgba(236, 72, 153, 0.4);}
.btn-outline {background: transparent;
    color: #1a202c;
    border: 1px solid #1a202c;}
.btn-outline:hover {background: #1a202c;
    color: white;}
.btn-sm {padding: 5px 10px;
    font-size: 12px;}
.btn-like {background: linear-gradient(135deg, #f8fafc 0%, #e2e8f0 100%);
    color: #475569;
    border: 1px solid #cbd5e1;}
.btn-like.liked {background: linear-gradient(135deg, #10b981 0%, #059669
100%);
    color: white;
```

```
    box-shadow: 0 4px 15px rgba(16, 185, 129, 0.4);}
.form-container {background: rgba(255, 255, 255, 0.95);
    -webkit-backdrop-filter: blur(10px);
    backdrop-filter: blur(10px);
    padding: 2.5rem;
    border-radius: 16px;
    box-shadow: 0 8px 32px rgba(0,0,0,0.1);
    border: 1px solid rgba(255, 255, 255, 0.2);
    max-width: 500px;
    margin: 0 auto}
.form-{groupmargin-bottom: 1rem;}
.form-{group labeldisplay: block;
    margin-bottom: 5px;
    font-weight: bold;}
.form-group input,
.form-group select,
.form-group textarea {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;}
.form-group textarea {
    resize: vertical;}
.alert {padding: 10px;
    margin-bottom: 1rem;
    border-radius: 4px;}
.alert-success {background: #d4edda;
    color: #155724;
```

```
    border: 1px solid #c3e6cb;}
.alert-error {background: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;}
.hero {text-align: center;
    background: rgba(255, 255, 255, 0.95);
    -webkit-backdrop-filter: blur(10px);
    backdrop-filter: blur(10px);
    padding: 3rem;
    border-radius: 20px;
    box-shadow: 0 8px 32px rgba(0,0,0,0.1);
    border: 1px solid rgba(255, 255, 255, 0.2);}
.hero h2 {margin-bottom: 1rem;
    color: #1a202c;}
.hero p {margin-bottom: 2rem;
    color: #4a5568;}
.buttons {display: flex;
    gap: 1rem;
    justify-content: center;}
.stories-grid {display: grid;
    grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
    gap: 1.5rem;}
.story-card {background: rgba(255, 255, 255, 0.95);
    -webkit-backdrop-filter: blur(10px);
    backdrop-filter: blur(10px);
    padding: 1.5rem;
    border-radius: 16px;
    box-shadow: 0 8px 32px rgba(0,0,0,0.1);
    border: 1px solid rgba(255, 255, 255, 0.2);
```

```
    transition: all 0.3s ease;}
.story-card:hover {transform: translateY(-8px);
    box-shadow: 0 12px 40px rgba(0,0,0,0.15);}
.story-card h3 {margin-bottom: 0.5rem;}
.story-card h3 a {text-decoration: none;
    color: #1a202c;}
.story-card h3 a:hover {color: #14b8a6;}
.story-meta {color: #64748b;
    font-size: 0.9rem;
    margin-bottom: 1rem;}
.story-excerpt {margin-bottom: 1rem;
    line-height: 1.5;}
.story-actions {display: flex;
    justify-content: space-between;
    align-items: center;}
.likes, .comments {color: #7f8c8d;
    font-size: 0.9rem;}
.story-detail {background: white;
    padding: 2rem;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    margin-bottom: 2rem;}
.story-detail header {border-bottom: 1px solid #eee;
    padding-bottom: 1rem;
    margin-bottom: 2rem;}
.story-detail h1 {color: #2c3e50;
    margin-bottom: 1rem;}
.story-content {line-height: 1.8;
    margin-bottom: 2rem;
```

```
    white-space: pre-wrap}
.comments-section {background: white;
    padding: 2rem;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);}
.comment-form {margin-bottom: 2rem;
    padding-bottom: 2rem;
    border-bottom: 1px solid #eee;}
.comment {padding: 1rem;
    border-bottom: 1px solid #eee;
    margin-bottom: 1rem;}
.comment:last-child {border-bottom: none;
    margin-bottom: 0;}
.comment-header {display: flex;
    justify-content: space-between;
    margin-bottom: 0.5rem;}
.comment-date {color: #7f8c8d;
    font-size: 0.9rem;}
.admin-sections {display: flex;
    flex-direction: column;
    gap: 2rem;}
.admin-section {background: white;
    padding: 2rem;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);}
.admin-table {width: 100%;
    border-collapse: collapse;
    margin-top: 1rem;}
.admin-table th,
```

```
.admin-table td {padding: 12px;
   text-align: left;
   border-bottom: 1px solid #ddd;}
.admin-table th {background: #f8f9fa;
   font-weight: bold;}
.admin-table tr:hover {background: #f8f9fa;}
@media (max-width: 768px) {
   .navbar { flex-direction: column;
      gap: 1rem;}
   .nav-links {margin-right: 0; }
   .buttons {flex-direction: column;
      align-items: center; }
   .stories-grid {grid-template-columns: 1fr; }
   .story-actions {flex-direction: column;
      gap: 0.5rem;
      align-items: flex-start; }}
```

TEMPLATES

Dashboard.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
   <meta charset="UTF-8">
   <title>Dashboard - Campus Mentorship Stories</title>
   <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
   <div class="container">
      <nav class="navbar">
```

```
        <h1>Campus Stories</h1>
        <div class="nav-links">
            <span sec:authentication="name"></span>
            <a href="/story/new" class="btn btn-primary">New Story</a>
                <a th:if="${currentUser != null and currentUser.role.name() == 'ADMIN'}" href="/admin/dashboard" class="btn btn-secondary">Admin</a>
            <form th:action="@{/logout}" method="post" style="display: inline;">
                <button type="submit" class="btn btn-outline">Logout</button>
            </form>
        </div>
    </nav>
    <main>
        <div class="stories-grid">
            <div th:each="story : ${stories}" class="story-card">
<h3><ath:href="@{/story/{id}(id=${story.id})}"
th:text="${story.title}"></a></h3>
 <p class="story-meta">
By: <span th:text="${story.authorName ?: story.author.username}"></span> |
Category: <span th:text="${story.category}"></span> |
<span th:text="${#temporals.format(story.createdAt, 'MMM dd, yyyy')}"></span>
</p><p class="story-excerpt" th:text="${#strings.abbreviate(story.content, 150)}"></p>
<div class="story-actions">
<span class="likes">❤️ <span th:text="${story.likeCount}"></span></span>
<span class="comments">💬 <span th:text="${#lists.size(story.comments)}"></span></span>
<div th:if="${currentUser != null and (story.author.id == currentUser.id or currentUser.role.name() == 'ADMIN')}">
```

```
form th:action="@{/story/{id}/delete(id=${story.id})}" method="post" style="display: inline;">
                                <button type="submit" class="btn btn-danger btn-sm" onclick="return confirm('Delete this story?')">Delete</button>
                            </form>
                        </div>
                    </div>
                </div>
            </div>
        </main>
    </div>
</body>
</html>
```

Index.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Campus Mentorship Stories</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <div class="container">
        <header>
            <h1>Campus Mentorship Stories</h1>
            <p>Share and discover inspiring stories from our campus community</p>
        </header>
        <main>
            <div class="hero">
```

```
            <h2>Welcome to Our Community</h2>
<p>Connect with students, teachers, and mentors through meaningful stories</p>
            <div class="buttons">
                <a href="/login" class="btn btn-primary">Login</a>
                <a href="/register" class="btn btn-secondary">Register</a>
            </div>
        </div>
    </main>
  </div>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Login - Campus Mentorship Stories</title>
  <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
  <div class="container">
    <div class="form-container">
      <h2>Login</h2>
      <div th:if="${param.registered}" class="alert alert-success">
        Registration successful! Please login.
      </div>
      <div th:if="${param.error}" class="alert alert-error">
        Invalid username or password.
      </div>
```

```
        <form th:action="@{/login}" method="post">
            <div class="form-group">
                <label for="username">Username:</label>
                <input type="text" id="username" name="username" required>
            </div>
            <div class="form-group">
                <label for="password">Password:</label>
                <input type="password" id="password" name="password" required>
            </div>
            <button type="submit" class="btn btn-primary">Login</button>
        </form>
        <p><a href="/register">Don't have an account? Register here</a></p>
        <p><a href="/">Back to Home</a></p>
    </div>
  </div>
</body>
</html>
```

Register.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Register - Campus Mentorship Stories</title>
  <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
  <div class="container">
    <div class="form-container">
      <h2>Register</h2>
```

```
        <div th:if="${error}" class="alert alert-error" th:text="${error}"></div>
        <form th:action="@{/register}" th:object="${user}" method="post">
            <div class="form-group">
                <label for="username">Username:</label>
                <input type="text" id="username" th:field="*{username}" required>
            </div>
            <div class="form-group">
                <label for="email">Email:</label>
                <input type="email" id="email" th:field="*{email}" required>
            </div>
            <div class="form-group">
                <label for="password">Password:</label>
        <input type="password" id="password" th:field="*{password}" required>
            </div>
            <div class="form-group">
                <label for="userType">User Type:</label>
                <select id="userType" th:field="*{userType}" required>
                    <option value="">Select Type</option>
                    <option value="STUDENT">Student</option>
                    <option value="TEACHER">Teacher</option>
                    <option value="MENTOR">Mentor</option>
                </select>
            </div>
            <button type="submit" class="btn btn-primary">Register</button>
        </form>
        <p><a href="/login">Already have an account? Login here</a></p>
        <p><a href="/">Back to Home</a></p>
    </div>
</div>
```

```
</body>
</html>
```

Story-Detail.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title th:text="${story.title} + ' - Campus Mentorship Stories'"></title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <div class="container">
        <nav class="navbar">
            <h1>Campus Stories</h1>
            <div class="nav-links">
                <a href="/dashboard" class="btn btn-secondary">Back to Dashboard</a>
            </div>
        </nav>
        <main>
            <article class="story-detail">
                <header>
                    <h1 th:text="${story.title}"></h1>
                    <div class="story-meta">
<p>By: <span th:text="${story.authorName ?: story.author.username}"></span></p>
                        <p>Category: <span th:text="${story.category}"></span></p>
                        <p>Published: <span th:text="${#temporals.format(story.createdAt, 'MMM dd, yyyy HH:mm')}"></span></p>
                    </div>
```

```
        </header>
        <div class="story-content" th:text="${story.content}"></div>
        <div class="story-actions">
          <form th:action="@{/story/{id}/like(id=${story.id})}" method="post"
style="display: inline;">
            <button type="submit" class="btn btn-like" th:class="${isLiked} ?
'btn btn-like liked' : 'btn btn-like'">
              ❤️ <span th:text="${story.likeCount}"></span>
            </button>
          </form>
          <div th:if="${currentUser != null and (story.author.id == currentUser.id
or currentUser.role.name() == 'ADMIN')}">
            <form  th:action="@{/story/{id}/delete(id=${story.id})}"
method="post" style="display: inline;">
              <button type="submit" class="btn btn-danger" onclick="return
confirm('Delete this story?')">Delete Story</button>
            </form>
          </div>
        </div>
      </article>
      <section class="comments-section">
        <h3>Comments</h3>
          <form  th:action="@{/story/{id}/comment(id=${story.id})}"
th:object="${newComment}" method="post" class="comment-form">
          <div class="form-group">
            <textarea th:field="*{content}" placeholder="Add a comment..."
rows="3" required></textarea>
          </div>
          <button  type="submit"  class="btn  btn-primary">Post
```

```
Comment</button>
            </form>
            <div class="comments-list">
               <div th:each="comment : ${comments}" class="comment">
                  <div class="comment-header">
                     <strong th:text="${comment.user.username}"></strong>
<span  class="comment-date"  th:text="${#temporals.format(comment.createdAt,
'MMM dd, yyyy HH:mm')}"></span>
                  </div>
                  <p th:text="${comment.content}"></p>
               </div>
            </div>
         </section>
      </main>
   </div>
</body>
</html>
```

Story-Form.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
   <meta charset="UTF-8">
   <title>New Story - Campus Mentorship Stories</title>
   <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
   <div class="container">
      <nav class="navbar">
         <h1>Campus Stories</h1>
```

```
        <div class="nav-links">
          <a href="/dashboard" class="btn btn-secondary">Back to Dashboard</a>
        </div>
      </nav>
      <div class="form-container">
        <h2>Share Your Story</h2>
        <form th:action="@{/story/save}" th:object="${story}" method="post">
          <div class="form-group">
            <label for="title">Title:</label>
            <input type="text" id="title" th:field="*{title}" required>
          </div>
          <div class="form-group">
            <label for="category">Category:</label>
            <select id="category" th:field="*{category}" required>
              <option value="">Select Category</option>
              <option value="Academic">Academic</option>
              <option value="Career">Career</option>
              <option value="Personal Growth">Personal Growth</option>
              <option value="Research">Research</option>
              <option value="Internship">Internship</option>
              <option value="Other">Other</option>
            </select>
          </div>
          <div class="form-group">
            <label for="authorName">Author Name (Optional):</label>
              <input type="text" id="authorName" th:field="*{authorName}"
placeholder="Leave blank to use username">
          </div>
          <div class="form-group">
```

```
            <label for="content">Story Content:</label>
<textarea id="content" th:field="*{content}" rows="10" required></textarea>
            </div>
            <button type="submit" class="btn btn-primary">Publish Story</button>
          </form>
      </div>
   </div>
</body>
</html>
```

Application.properties

```
spring.application.name=cmapus_blog
# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/campus_blog?
createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=nisanthu@2006
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
# Thymeleaf Configuration
spring.thymeleaf.cache=false
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project                              xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
<parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>3.5.7</version>
   <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.campus_blog</groupId>
<artifactId>cmapus_blog</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>cmapus_blog</name>
<description>Demo project for Spring Boot</description>
<url/>
<licenses>
   <license/>
</licenses>
<developers>
   <developer/>
</developers>
<scm>
   <connection/>
   <developerConnection/>
   <tag/>
   <url/>
</scm>
<properties>
   <java.version>21</java.version>
</properties>
<dependencies>
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
```

```
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

ANNEXURE I

PROJECT TITLE	**Urban Automated Underground Waste Pneumatic Transport Hub**
PROGRAM	**B.E. COMPUTER SCIENCE & ENGINEERING**
PROJECT BATCH NUMBER	**21**
BATCH MEMBERS	**722825104234 THIRULOGESH M**
NAME OF THE SUPERVISOR	**Mr. M. KARTHICK RAJA M.E.,**
NAME OF THE SDG GOALS MAPPED	Industry, Innovation and Infrastructure, Sustainable Cities and Communities, Responsible Consumption
MENTION THE SDG GOALS NUMBER	**SDG 9, SDG 11, SDG 12**
NAME OF THE TRL LEVEL	**Experimental Proof of Concept**
MENTION THE TRL LEVEL	**TRL 3**

POs & PSOs Mapping (Put a tick mark in the mapped PO's & PSO's):

Program Outcomes											Program Specific Outcomes		
PO1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓

Signature of the Supervisor

Mr. M. Karthick Raja M.E.,

ANNEXURE II



VENUE AND EXPENDITURE STATEMENT FOR THE PROJECT WORK

Laboratory details where the project is carried out	**Full stack Lab, CSE**
Software / Hardware details	**SOFTWARE:** **Spring Boot 3.1.5, Spring Security 6.1, Spring Data JPA, Spring Web MVC, MySQL 8.0, Hibernate ORM, MySQL Connector/J 8.1.0, Thymeleaf 3.1, HTML5, CSS3, JavaScript ES6, Apache Maven 3.9, Java JDK 17.** **HARDWARE: Nil**

Details of the Component and Expenditure

S. No	Name of the Component	Qty	Price / Unit in (Rs.)	Amount (Rs.)
1.	Spring Boot Framework	1	-	Open source
2.	MySQL Database	1	-	Open source
3.	Java JDK 17	1	-	Open source
4.	Apache Maven	1	-	Open source
5.	Thymeleaf Template Engine	1	-	Open source
6.	Spring Security	1	-	Open source
7.	VS Code	1	-	Open source
8.	Git Version Control	1	-	Open source
9.	MySQL Workbench	1	-	Open source

Signature of the student
NISANTHU. P

Signature of the Supervisor
Mr. M. KARTHICK RAJA M.E.,